Лабораторна робота № 2

з дисципліни «Системне програмування»

на тему «Інтеграція сенсорної підсистеми ParcelLog»

Виконав:

студент групи ІО-33

Чекун Микола Олексійович

Перевірив:

Гордієнко Нікіта Юрійович

Київ – 2026

Тема: Інтеграція сенсорної підсистеми ParcelLog: підключення, аналіз інтерфейсів і шарова архітектура.

Мета роботи: Підключити всі три сенсори одночасно, підтвердити їхню роботу незалежними цифровими записами, перевірити portable drivers та прикладні правила тестами, визначити межі симуляційної перевірки.

Постановка завдання

За методичкою ЛР2 необхідно обґрунтувати pinout і бюджет живлення, зібрати спільний стенд, виконати ідентифікацію й самоперевірку, записати I2C/UART/GPIO та маркер ADC. Потрібно декодувати ID і burst, вручну розібрати адресні біти, створити NACK, виміряти час, відокремити vendor/board/device/application layers, додати fake tests та власний captured vector, виміряти coverage й налаштувати CI. Результати мають містити component diagram, журнал проблем і межі достовірності.

Хід роботи

Призначення та межі перевірки

ParcelLog спостерігає за посилкою під час перевезення й зберігання. Система реєструє механічний вплив, зміну освітлення та рух поруч. Ці сигнали не встановлюють особу людини, її намір або факт відкриття упаковки. Використано форму B: модель NUCLEO-C031C6, MPU6050, LDR і PIR в одному стенді Wokwi. Середовище: Ubuntu 24.04, Docker, STM32 HAL, Wokwi.

Первинний PRD з ЛР1 збережено. Режими transit і storage обираються фізичним перемикачем на PB0; PIR на PB1 враховується лише у storage. Всі записи отримані 15.10.2026. Час захоплення й журнали виконання не підмінюються датами комітів. Перевірка стосується цифрової моделі; фізична плата не випробовувалась. Камера, SPI-flash і PWM не входять до цієї конфігурації. UART-журнал існує лише до перезавантаження, постійного сховища немає.

Архітектурні рішення

Core налаштовує тактування, alternate functions та IRQ. Platform board містить апаратний I2C, ADC і UART, а Platform runtime планує опитування, обробляє команди й читає режим. Devices реалізує MPU6050 через callbacks SensorBus, без HAL includes. App містить переносні правила подій та працює із сирими відліками прискорення. Поряд із raw драйвер обчислює фізичні одиниці для журналу й тестів. Так зберігаються точні пороги початкового PRD без подвійного округлення.

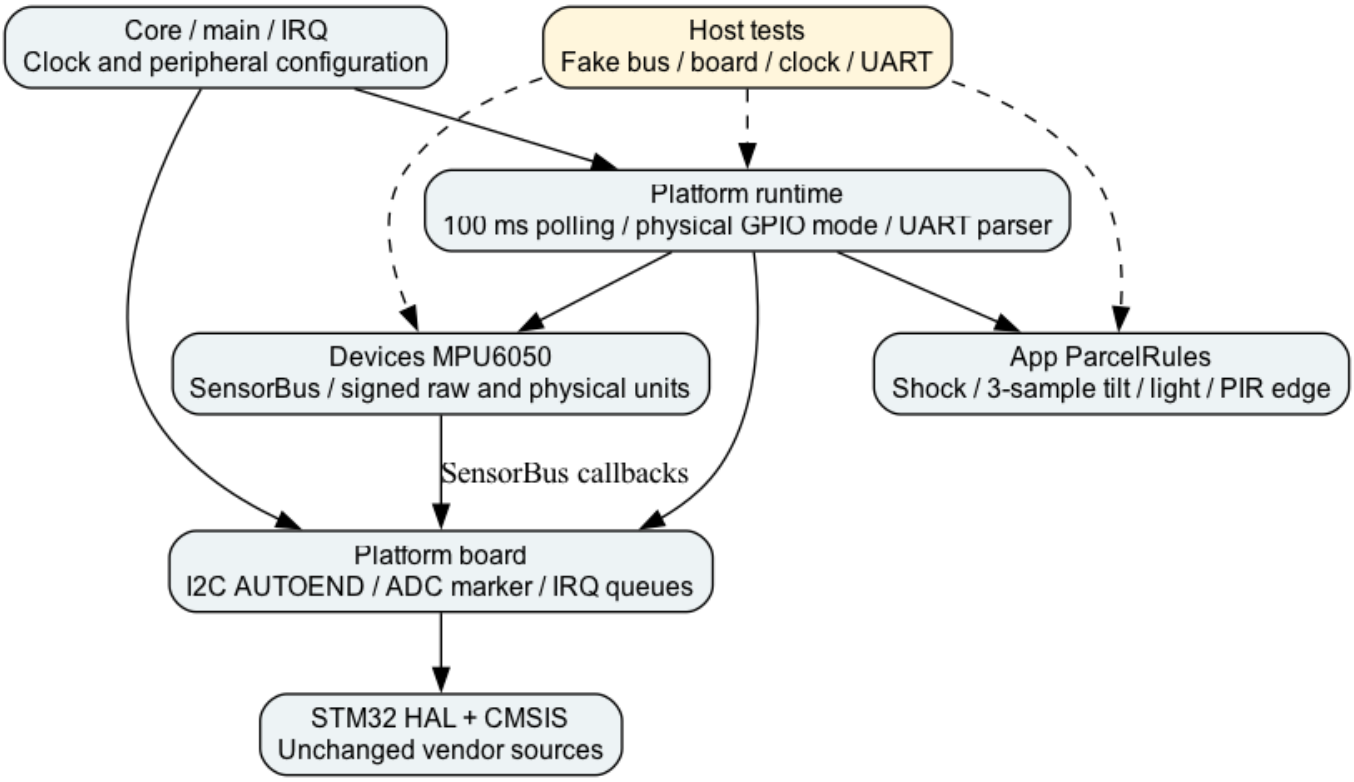


Рисунок 1 – Компоненти ParcelLog та напрямки залежностей

Pinout і перевірка конфліктів

Розкладку зіставлено з DS13867 Rev4, розділом 4 і таблицями alternate functions портів A/B, а маршрути плати з UM2953 Rev1, рис.7 с.19, табл.11 с.20 та VCP §6.9 с.16. Для плати перевіряються саме MCU NUCLEO-C031C6 та роз'єми MB1717, а не розкладка іншої STM32 Nucleo. Позначення D14/D15 не замінюють номер MCU pin.[1, 2]

MCU	Функція	З'єднання та конфлікти
PB8 / PB9	I2C1 SCL/SDA, AF6	MPU SCL/SDA; pull-up 4,7 кОм до 3,3 V. D15/D14, CN5-10/9.
PA0	ADC1 IN0	Аналоговий вихід LDR, A0, CN8-1. D0 не під-ключено.

MCU	Функція	З’єднання та конфлікти
PB0	GPIO input	Контакт 2 перемикача; крайні контакти GND і 3,3 V. D10, CN5-3.
PB1	GPIO input	PIR OUT, A3, CN8-4; внутрішній pull-down.
PA2 / PA3	USART2 TX/RX, AF1	Serial monitor та RFC2217; PA2 CN7-37, PA3 CN10-10, VCP.
PA4	GPIO output	ADC_MARK до D4 аналізатора; A2, CN8-3.
PA5	GPIO output	Бортовий LED LD1, D13; додатковий load не підключено.

D0/D1 належать USART1 PB7/PB6, їх не можна підміняти USART2. У табл.12 с.23 UM2953 підписи SDA/SCL біля PB8/PB9 переставлено відносно табл.11; використано DS13867 і табл.11: PB8=SCL, PB9=SDA. Номери роз’ємів наведено за UM2953; саме MCU pin у першому стовпці визначає з’єднання diagram.json.

Всі компоненти мають спільний GND. У моделі MPU, LDR і PIR живляться від 3,3 V. Для реального PIR спочатку потрібна специфікація конкретного модуля: зокрема, модель не доводить, що довільний модуль із регулятором працюватиме від 3,3 V. Рівень OUT має бути сумісним із 3,3 V MCU; для 5 V OUT потрібен перетворювач. MPU AD0 приєднано до GND, INT не використано. SDA/SCL є open-drain; SWD PA13/PA14 вільні. USART2 використовує VCP, тому перед апаратним запуском треба звірити solder bridges і виключити двох активних передавачів на RX. DMA transfers немає, отже конфлікту DMA requests у поточній схемі немає. Таймери й PWM не використовуються.

Живлення та електричні межі

За специфікацією MPU6050 VDD становить 2,375–3,46 V, цифрові рівні залежать від VLOGIC.[3] Для Wokwi дані про спожитий струм відсутні. Як консервативний попередній бюджет беремо MCU 15 mA, MPU 5 mA, LDR module 5 mA, PIR module 1 mA і LED 4 mA: разом 30 mA. Це розрахункові резерви, а не вимірювання конкретних модулів. Окремо треба врахувати ST-LINK та живлення плати.

Для одного pull-up 4,7 кОм при LOW струм приблизно (3,3-0,4)/4700=0,62 mA. За припущення C=100 pF маємо $t_r \approx 0{,}8473RC = 0{,}398$ мкс; це лише оцінка RC, а ідеальні фронти симулятора її не підтверджують. На фізичному стенді слід виміряти 3,3 V і піки струму, перевірити фактичні резистори та фронти осцилографом, встановити

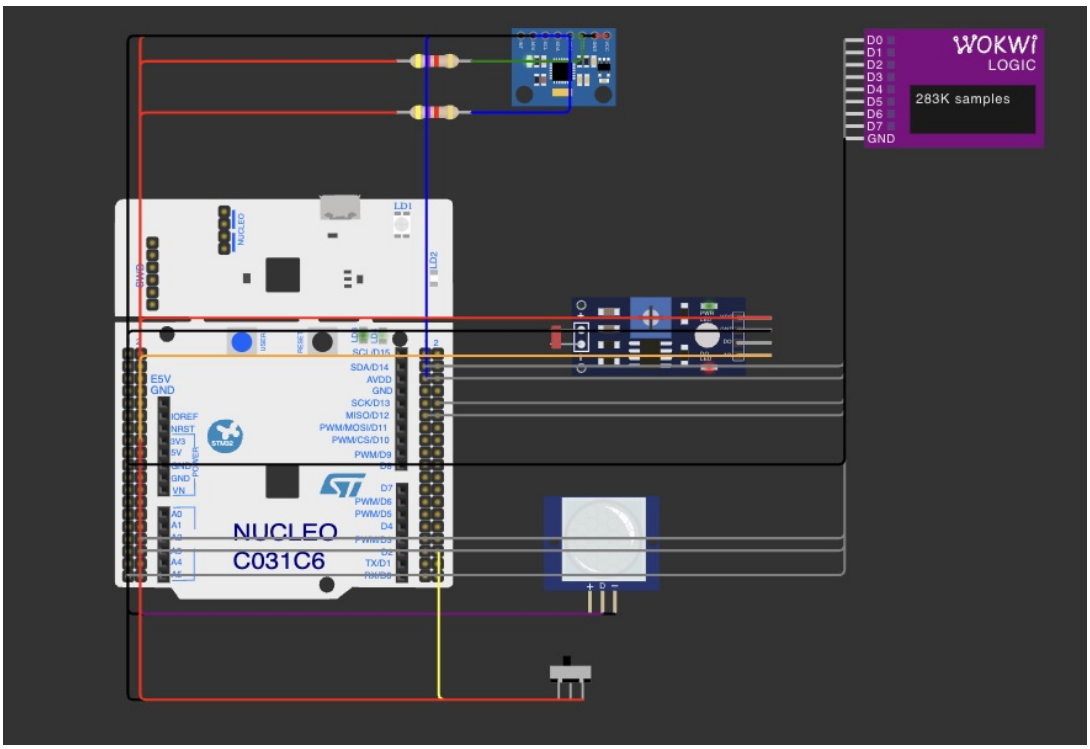


Рисунок 2 – Спільний стенд ParcelLog із перемикачем режиму та аналізатором

розв’язувальні конденсатори біля модулів.

Тактування і сенсорний протокол

HSI, SYSCLK, HCLK та PCLK налаштовано на 48 МГц. I2C1 TIMINGR=0x20303E5D, PRESC=2, SCLH=62, SCLL=93. Спрощена оцінка $(62 + 93 + 2) \cdot 3/48$ МГц дає 9,8125 мкс без затримок фільтрів. За фронтами SCL фактично одержано 10 мкс, тобто 100 кГц. MCU reference manual визначає повний розрахунок із затримками синхронізації та фільтрів.[4]

MPU6050 працює за 7-бітною адресою 0x68. Команда selftest читає WHO_AM_I з регістра 0x75 і перевіряє 0x68, потім записує 0x6B=1, 0x19=9, 0x1A=3, 0x1B=0 і 0x1C=0. Вибрано PLL, дільник 10, DLPF та діапазони $\pm 2g$ і $\pm 250^\circ/\text{c}$. За DLPF output 1 кГц це дає внутрішнє оновлення 100 Гц; застосунок читає кожні 100 мс.[5]

Burst читає 14 байтів від 0x3B, усі осі прискорення, температуру й gyro. У спокої незалежний запис дає:

```
00 00 00 00 40 00 EF 5C 00 00 00 00 00 00
```

Сирий Z=0x4000=16384 відповідає 1000 mg; норма вектора дорівнює 1 g. Температура

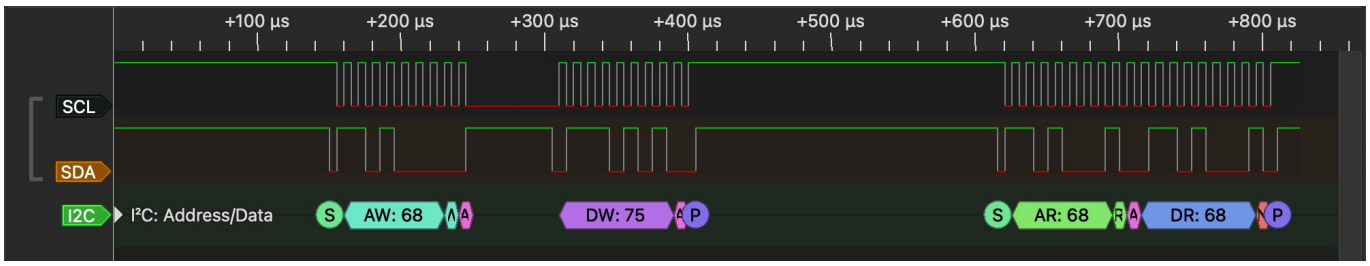


Рисунок 3 – WHO_AM_I: адреса 0x68, pointer 0x75, дані 0x68

0xEF5C=-4260 дає $-4260 \cdot 1000/340 + 36530 = 24001$ m°C. Вхід моделі 24°C; різниця 1 m°C спричинена цілочисловим округленням. За впливу X=2 g при Z=1 g модель дає X=32767, тобто 1999 mg. Це ступінчастий тест, а не фізичний чистий нахил, оскільки норма вектора стає більшою за 1 g.

```
static int32_t signed_word(const uint8_t *data) {
    uint32_t word = ((uint32_t)data[0] << 8) | data[1];
    return word < 32768u ? (int32_t)word : (int32_t)word - 65536;
}

void mpu6050_decode(const uint8_t bytes[14], ImuSample *sample) {
    for (size_t axis = 0; axis < 3; ++axis) {
        sample->accel_raw[axis] = (int16_t)signed_word(bytes + 2 * axis);
        sample->accel_mg[axis] = signed_word(bytes + 2 * axis) * 1000 /
   16384;
        sample->gyro_mdps[axis] = signed_word(bytes + 8 + 2 * axis) * 1000 /
   131;
    }
    sample->temperature_mc = signed_word(bytes + 6) * 1000 / 340 + 36530;
}
```

Запис і ручне декодування

Wokwi logic analyzer записує всі вісім ліній одночасно: D0 SCL, D1 SDA, D2 UART TX, D3 PIR, D4 ADC marker, D5 LED, D6 UART RX та D7 MODE. Тригер починається із симуляцією; у docs/captures є повний VCD без втрат, дев’ять вікон VCD/SR/PVS, source SHA256 і початковий offset. Для I2C/UART/ADC detail використано SR 20 МГц, що дає 200 вибірок на період I2C; для довгих GPIO вікон 1 МГц. Квантування SR відповідно 50 нс або 1 мкс.[6, 7]

На сирій формі START є спадом SDA за HIGH SCL. На восьми фронтах SCL бачимо 1 1 0 1 0 0 0 0, тобто 0xD0. Старші сім біт 1101000 означають адресу 0x68, R/W=0

означає запис. Дев’ятий біт SDA=0 є ACK від MPU. Наступні біти 01110101 є register pointer 0x75.

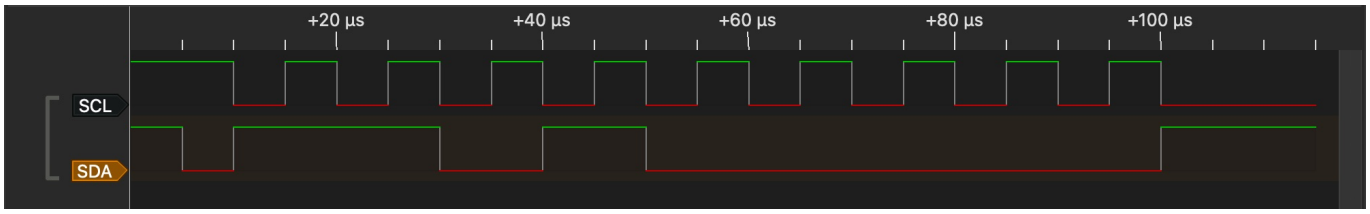


Рисунок 4 – Адресний байт без протокольних анотацій

Повна pointer+burst операція займає 1789.375 мкс; медіанний період burst 99.99565 мс. Читання використовує STOP між pointer і read; MPU зберігає pointer. Останній байт читання завершує master NACK, який є штатним завершенням, а не відмовою адреси.

Контрольований NACK і відновлення

Команда imuaddr 0x69 змінює адресу драйвера при AD0=0. Байтові 0xD2 відповідає непідтверджена адреса 0x69; дев’ятий біт SDA=1. Ця операція триває 105.0 мкс. На 99233 мс selftest дав result=1; повтори на 101306 і 103306 мс підтверджують retry приблизно через 2 с. Освітлення, UART tick та LED працювали далі. Після imuaddr 0x68 на 104234 мс WHO_AM_I й ініціалізація відновились. Старі сирі значення IMU не видаються як актуальні: status позначає error та нулі.

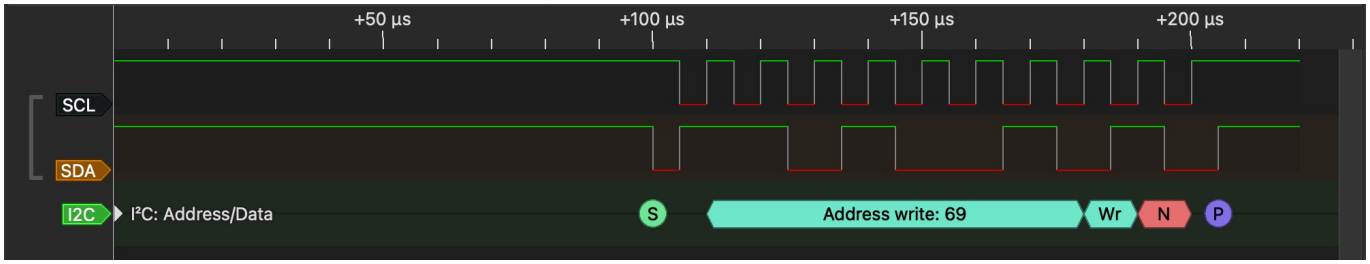


Рисунок 5 – Адресний slave NACK на 0x69

I2C adapter обмежує операцію deadline 10 мс, перевіряє NACK/BERR/ARLO/OVR, очищує flags і CR2, при помилці перевмикає PE. Замість GPIO bit-bang із ЛР1 використано апаратний I2C1. Цілі записи CR2 та паузи 120 мкс після START/TX/STOP узгоджують модель Wokwi. Це специфічна для перевіреного шляху адаптація; її час і потребу треба переоцінити на фізичній платі.

UART, ADC і GPIO

USART2 працює на 115200 8N1, теоретичний біт 8,6806 мкс, кадр 86,806 мкс. Сигнал TX декодовано незалежно від terminal log. ASCII початок seq=1 наведено нижче, а повні рядки з часовими мітками збережено в wire-uart.txt.

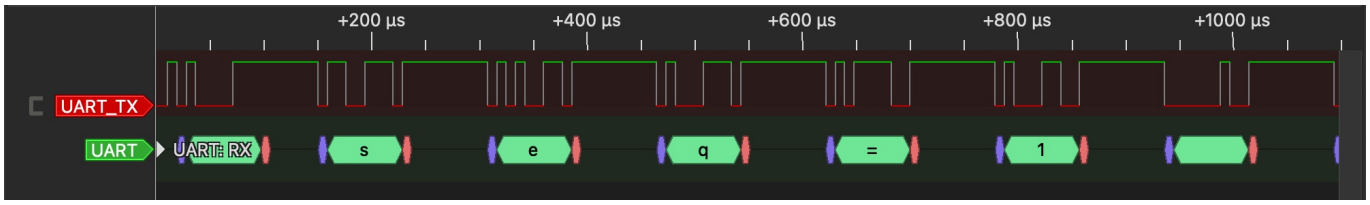


Рисунок 6 – ASCII-декод рядка події з USART2

TX та RX мають IRQ queues на 2048 і 256 байтів. Переповнення відстежується. Парсер приймає status, selftest, imuaddr 0x68 та imuaddr 0x69; інші рядки, NUL або понад 63 символи відкидаються до нового рядка. За один poll обробляються щонайбільше 128 символів. Режим не змінюється UART-командою: він визначається тільки PB0. Жоден основний цикл не містить HAL_Delay; операції I2C й ADC синхронні, але обмежені timeout.

ADC1 IN0 працює з PCLK/2=24 МГц, 12-бітною роздільністю й 160,5 sampling cycles. Перед першим читанням виконується HAL ADC calibration. Розрахунковий час вибірки та перетворення $(160{,}5+12{,}5)/24$ МГц близько 7,21 мкс. PA4 огортає Start/Poll/Get/Stop: виміряна ширина 65.225 мкс включає software та модель і не є апертурою ADC. Медіанний період маркера 99.99565 мс відповідає плану 100 мс.

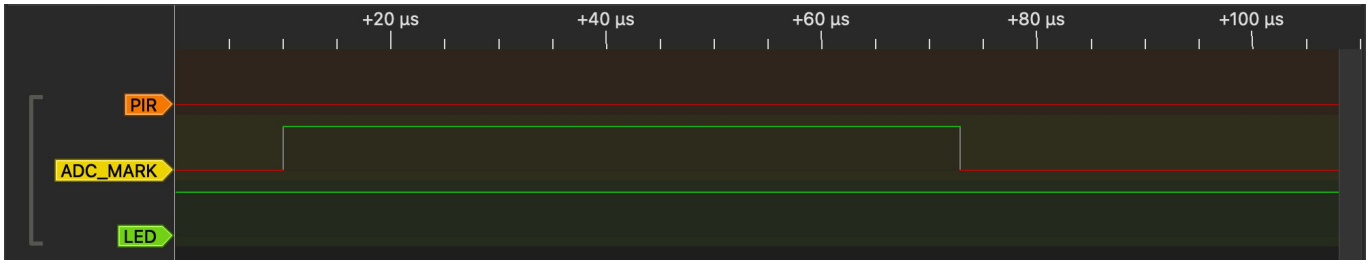


Рисунок 7 – Маркер програмного вимірювання ADC на PA4

При 500 lux модель LDR дала 1001 відлік, після 100000 lux 32. Обидва значення в діапазоні 0–4095, напрям зміни очікуваний для дільника. Перерахунок ADC у калібровані lux не реалізовано. У разі помилки ADC канал повторно ініціалізується через 1 с;

IMU і PIR працюють незалежно. PIR HIGH тривав 3.000000025 с. LED перемикається кожні 500 мс, повний період приблизно 1 с; він є heartbeat, а не тривожним силовим виконавцем.

Правила подій та одночасний тест

Shock виникає, коли найбільший модуль однієї осі не менше 27853 raw, близько 1,7 g. Переозброєння вимагає спаду нижче 22937 raw, близько 1,4 g. Це максимум осей, а не норма вектора. Tilt вимагає трьох послідовних валідних вимірів X або Y не менше 10649 raw, близько 0,65 g; скидання нижче 8192 raw по обох бокових осях. Пропуск IMU або вимір у проміжній зоні перериває непідтверджену серію. Активна подія зберігає гістерезис до нормального виміру.

Початкове світло стає reference; зміна не менше 500 raw дає light_change, повернення до різниці менш як 300 переозброює детектор. Цифрового averaging тут немає. PIR дає nearby_motion лише на фронт у storage. HIGH при вході в storage не створює нової події. У transit імпульс не дає подію, але актуальний рівень запам’ятовується.

У паузі симуляції встановлено X=2 g при Z=1 g, LDR=100000 lux і натиснуто Simulate motion, після чого виконання продовжено. Режим storage був увімкнений раніше. На 47205 мс в одному циклі виникли shock, light_change та nearby_motion, а на 47405 мс tilt. Від першого до третього 100-мс виміру минає 200 мс; твердження про 300 мс між цими часовими мітками було б неправильним. Нумерація seq зростає окремо для кожної події, утримання впливу не створює повторів.

```
seq=2 time_ms=46305 event=mode_changed mode=storage
tick [47001 ms] mode=storage imu=ok raw=0,0,16384 mg=0,0,1000 gyro=0,0,0 temp_mc=24001 light=1001 pir=0 tx_overflow=0
seq=3 time_ms=47205 event=shock mode=storage
seq=4 time_ms=47205 event=light_change mode=storage
seq=5 time_ms=47205 event=nearby_motion mode=storage
seq=6 time_ms=47405 event=tilt mode=storage
tick [48001 ms] mode=storage imu=ok raw=32767,0,16384 mg=1999,0,1000 gyro=0,0,0 temp_mc=24001 light=32 pir=1 tx_overflow=0
tick [49001 ms] mode=storage imu=ok raw=32767,0,16384 mg=1999,0,1000 gyro=0,0,0 temp_mc=24001 light=32 pir=1 tx_overflow=0
```

Рисунок 8 – Реальні події та сирі покази ParcelLog

Бібліотеки, збірка і тести

CMSIS та STM32C0 HAL з початкового репозиторію збережено. HAL source files не змінювались. RCC/FLASH налаштовують clock, CORTEX/PWR підтримують core,

GPIO/EXTI керують pin, USART працює з IRQ, I2C задає периферію, ADC/ADCEx виконують аналогові виміри. DMA лишився у вендорському наборі, але transfers не використовуються. tools/check_project.py забороняє залежність App/Devices від HAL і board. Docker base image закріплено digest; build використовує ARM GCC, -Os, -Wall та -Wextra. Власний код збирається без попереджень.

Три assert-based suites перевіряють MPU, ParcelRules та runtime з fake шиною, платою, clock і UART. Для драйвера перевірено ID, усі п’ять config writes, NACK, timeout і error, signed limits і незмінність output за відмови. Тести правил охоплюють одночасні події, гістерезис, межу наклону, INT16_MIN, пропуск IMU та GPIO mode entry. Runtime tests перевіряють незалежність ADC при NACK, повторну спробу ADC, CRLF/довгі команди та wraparound часу. Окремо два fixture взято із власного VCD із SHA256 та часом транзакції. Вони дають ті самі raw та фізичні одиниці, що й UART.

GitHub Actions на branch/PR запускає firmware build, Linux tests з ASan і UBSan та coverage. Пороги 85% рядків і 70% гілок стосуються тільки переносних App і Devices. Одержано 74/74 рядків (100.0%) і 54/56 гілок (96.43%). Board/HAL не входять у цей відсоток; runtime перевірений fake-тестом окремо. Цифрові апаратні шляхи перевірені Wokwi, фізичне обладнання не перевірено.

Журнал виправлень і відповідальність

Виявлено помилку серії tilt: два високі виміри, відмова IMU та ще один високий вимір раніше давали подію. Тепер invalid і проміжна зона переривають серію; регресійні тести це перевіряють. GPIO bit-bang замінено обмеженим апаратним I2C. ADC доповнено калібруванням та recovery, блокуючий UART передавач замінено IRQ queue, а HAL_Delay прибрано. Зміни не підмінюють початкову семантику PB0/PB1.

Всі напрями реалізував Чекун М.О.: вимоги та звіт; pinout і живлення; стенд і записи; PulseView та декодування; архітектура і збірка; тести й coverage. Артефакти розміщені у docs/lab2, docs/captures і tests. Репозиторій: https://github.com/chekunm/ParcelLog, гілка feature/lab-2.

Висновки

Інтегровано MPU6050, LDR і PIR разом із фізичним перемикачем режиму. Незалежний запис підтвердив WHO_AM_I=0x68, 100-кГц I2C, 100-мс опитування IMU й ADC, UART 115200 8N1 та очікувані сирі покази. Shock, світло й PIR не маскують один одного; tilt виникає після трьох послідовних вимірів. Створений slave NACK не зупиняє решту каналів, відновлення адреси повертає IMU в роботу.

Переносні правила та драйвер відокремлено від HAL; тести й coverage підтверджують обробку відмов і граничних значень. Виправлено розрив серії наклону та додано recovery ADC. Цифрова модель не підтверджує живлення реального PIR, аналогову точність, струм і якість фронтів на монтажній платі. Для фізичного варіанта потрібні окремі електричні та метрологічні вимірювання. UART-події не доводять відкриття посилки і не зберігаються після reset.

Список використаних джерел

[1] STMicroelectronics. *DS13867 Rev4. STM32C031x4/x6 datasheet*. URL: https://www.st.com/resource/en/datasheet/stm32c031g6.pdf.

[2] STMicroelectronics. *UM2953 Rev1. STM32 Nucleo-64 boards MB1717*. URL: https://www.st.com/resource/en/user_manual/um2953-stm32-nucleo64-board-mb1717-stmicroelectronics.pdf.

[3] InvenSense. *MPU-6000/MPU-6050 Product Specification, Rev3.4*. 2013. URL: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf.

[4] STMicroelectronics. *RM0490 Rev5. STM32C0x1 reference manual*.

[5] InvenSense. *MPU-6000/MPU-6050 Register Map and Descriptions, Rev4.2*. 2013. URL: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Register-Map1.pdf.

[6] Wokwi. *Logic Analyzer and VS Code Project Configuration*. URL: https://docs.wokwi.com/guides/logic-analyzer (дата зверн. 15.10.2026).

[7] sigrok project. *PulseView and protocol decoders*. URL: https://sigrok.org/wiki/PulseView (дата зверн. 15.10.2026).